Audio-Reactive Visual Engine: Design Plan

Nov 9, 2026 · @q

Vision

A performance-grade visual engine that listens to music and turns it into live visuals across every style: shaders, 3D, minimal and glitch. You can play it by hand, or leave it to run itself. It's built for you first; if it proves good, it goes public later.

Topic	Your answer
Core idea	Sound drives visuals, plus live VJ performance
Audience	You first, others later
Audio in	Uploaded songs and live microphone or line-in
Output	Live fullscreen, exported video, or just enjoying it
Styles	All of them: psychedelic shaders, 3D, clean/minimal, retro/glitch
Control	Presets, layers and nodes together, consistent with each other
Live input	Mouse and keyboard, MIDI controllers, fully automatic
Platform	Whatever suits the goal best (decided in the Platform section)

Best-of features

No single tool does everything you want. Each leader is best at one thing, so we take that one thing from each and join them under one control model (next sections).

What we take	Borrowed from	How it works there
Any parameter can follow the music	Resolume	Pick an FFT band on any slider and it bounces to the sound
Fast and slow versions of each band	MilkDrop	Presets read bass, mid and treble, plus smoothed bass_att, mid_att, treb_att

What we take	Borrowed from	How it works there
Maths-driven presets with feedback	MilkDrop / projectM	Per-frame and per-pixel equations warp the previous frame, giving endless trails
Shader scenes with standard audio inputs	Synesthesia	GLSL scenes, simple MIDI-mappable controls, imports from Shadertoy and ISF
A portable effect format	ISF (VDMX)	A shader plus a JSON header declaring its controls, multi-pass and persistent buffers; read by many VJ apps
Chain sources, modulators and feedback	Hydra	Analog-synth-style chains where a buffer can feed itself
Node networks with audio as channels	TouchDesigner	Audio analysis outputs channels you wire to anything
Layers, clips, BPM sync, one dial for many settings	Resolume	Decks and layers for live mixing; a Dashboard links many parameters to one knob
Tempo sync with DJs and DAWs	Ableton Link	Open network protocol for sharing tempo and beat phase; anyone can change the tempo
Send visuals to OBS and other apps	Spout / Syphon / NDI	Spout (Windows) and Syphon (Mac) share GPU frames on one machine; NDI sends video over a network
MIDI and OSC control	Synesthesia, Hydra	Map hardware knobs and pads to any control
Auto-pilot	Resolume, MilkDrop	Cycles clips or presets automatically

What we add that none of them do well:

1. **Knows the song's structure.** Instead of switching on a timer, auto-pilot detects builds, drops and phrase boundaries (every 8, 16 or 32 bars) and transitions on them.
2. **One modulation system everywhere.** Every parameter in a preset, a layer or a node works the same way, so the three ways to control it never feel like three different apps.

3. **Records the performance.** A live set saves as a timeline of what you did, so you can re-render it as a clean high-resolution video afterwards.

Licence note: Ableton Link's code is free under GPL, or under a paid licence for closed-source apps. If this stays open source it's free to use; if it goes commercial and closed, we contact Ableton first.

The maths

Every visual is driven by one small set of signals: band levels, onsets, beat phase and song structure. Each is computed below about 94 times a second and normalised so that **1.0 means typical for this song**. Any visual can then use them without per-song tuning.

1. Spectrum

Take 2048-sample windows of audio at 48 kHz (about 43 ms), stepping 512 samples each time (about 10.7 ms). Apply a Hann window, then an FFT:

```
X_n[k] = \sum_{m=0}^{N-1} w[m]\, x[nH+m]\, e^{-j 2\pi k m / N}, \qquad w[m] = \tfrac{1}{2}\left(1-\cos\tfrac{2\pi m}{N-1}\right)
```

Compress the magnitudes the way ears do, so quiet detail still registers:

```
Y_n[k] = \log\left(1 + \gamma\, |X_n[k]|\right), \qquad \gamma \approx 100
```

2. Bands on a hearing scale

Group the bins on the mel scale, which spaces pitch the way we hear it:

```
\mathrm{mel}(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right)
```

This gives 32 fine bands for detailed visuals, plus six named bands for everyday use: sub (20 to 60 Hz), bass (60 to 250 Hz), low-mid (250 to 500 Hz), mid (500 Hz to 2 kHz), high-mid (2 to 6 kHz) and air (6 to 16 kHz). Each band's energy is the mean power of its bins:

```
E_b(n) = \frac{1}{|B_b|} \sum_{k \in B_b} |X_n[k]|^2
```

3. Auto-gain: 1.0 means normal

Divide each band by its own slow running average. A quiet acoustic track and a loud techno track then drive visuals equally. MilkDrop uses the same idea.

$$\bar{E}_b(n) = \bar{E}_b(n-1) + \alpha_{\text{slow}}\left(E_b(n) - \bar{E}_b(n-1)\right), \qquad \ell_b(n) = \frac{E_b(n)}{\bar{E}_b(n) + \varepsilon}$$

4. Attack and release smoothing

Raw levels flicker. A one-pole smoother with separate attack and release times reacts instantly to hits but falls gently:

$$s_n = s_{n-1} + \alpha\,(x_n - s_{n-1}), \qquad \alpha = 1 - e^{-\Delta t / \tau}, \qquad \tau = \begin{cases} \tau_{\text{attack}} & x_n > s_{n-1} \\ \tau_{\text{release}} & \text{otherwise} \end{cases}$$

Every band gets two versions, like MilkDrop's bass and bass_att: **fast** (5 ms attack, 120 ms release) for punchy hits and **slow** (300 ms attack, 1.5 s release) for mood. Basing alpha on elapsed time keeps the feel identical at 30, 60 or 144 frames per second.

5. Onsets: hits, kicks, snares, hats

Spectral flux sums only the increases in the compressed spectrum, so new sounds stand out and fading ones are ignored:

$$\mathrm{SF}(n) = \sum_{k} \max\left(0,\; Y_n[k] - Y_{n-1}[k]\right)$$

An onset fires when the flux is a local peak above an adaptive threshold, with at least 50 ms between onsets:

$$\delta(n) = \lambda \cdot \operatorname{median}\{\mathrm{SF}(n-M), \dots, \mathrm{SF}(n)\} + \kappa$$

Running this per band gives separate kick (sub and bass), snare (low-mid and mid) and hat (air) triggers. The SuperFlux variant adds a small max filter across frequency, which cuts false hits from vibrato; it's an upgrade for later.

6. Tempo

Autocorrelate the onset signal over the last 6 seconds. Weight it towards common tempos so the detector doesn't lock onto half or double time:

$$R(\tau) = \sum_{n} \mathrm{SF}(n)\,\mathrm{SF}(n-\tau), \qquad W(\tau) = \exp\left(-\frac{\left(\log_2(\tau / \tau_{120})\right)^2}{2\sigma^2}\right), \qquad \tau^{*} = \arg\max_{\tau}\, W(\tau)\,R(\tau)$$

Here tau_120 is the lag equal to 120 BPM, and BPM = 60 / (tau* x hop time).

7. Beat phase: a clock that locks to the music

A phase counter runs from 0 to 1 every beat. Each detected onset nudges it slightly towards the true beat, the way a musician tightens up while playing along:

$$\varphi \leftarrow \varphi + \frac{\mathrm{BPM}}{60}\,\Delta t, \qquad \text{on onset: } \varphi \leftarrow \varphi + k\,\operatorname{wrap}(\varphi_{\text{onset}} - \varphi), \qquad k \approx 0.1$$

Counting beats gives **bar phase** (4 beats) and **phrase phase** (8, 16 or 32 bars). When Ableton Link is connected, its tempo and phase override this, so a DJ's mixer becomes the master clock.

8. Song structure: builds and drops

- **Build:** the slope of slow loudness over the last 8 bars, by linear regression, stays positive while the hats and onset rate rise.
- **Drop:** at a phrase boundary right after a build, slow bass jumps by a large ratio, about 2x the level before.
- **New section:** the cosine distance between the average spectrum of the last 4 bars and the 4 bars before it jumps.

These three events are what the auto-pilot listens for.

9. The mapping equation: one rule for every parameter

Every parameter in the app is a base value plus a sum of modulations, each one shaped, scaled and clamped to the parameter's range:

$$p(t) = \operatorname{clamp}\left(p_0 + \sum_{i} d_i \cdot S_i\left(m_i(t)\right),\; p_{\min},\; p_{\max}\right)$$

The source m(t) can be any signal above, an LFO locked to beat phase, a MIDI knob or a random walk. The shapes S cover most musical needs:

Shape	Formula	Use
Curve	x to the power g	Make a response gentler or punchier
Beat envelope	exp(-(t - t_hit) / decay)	A flash or kick on every hit
Phase pulse	(1 - phase) to the power k	A pumping effect locked to the beat, even between onsets

Shape	Formula	Use
Synced LFO	sin(2 pi x rate x bar phase)	Smooth motion that always lands on the bar
Step	floor(n x x) / n	Snappy, quantised changes

10. Two design rules of our own

Drive speed, not position. Feeding audio straight into position makes visuals jitter. Instead, audio sets a speed and the position accumulates it. The motion surges on loud passages but never jumps backwards. Several classic MilkDrop presets use this trick by accumulating a value every frame.

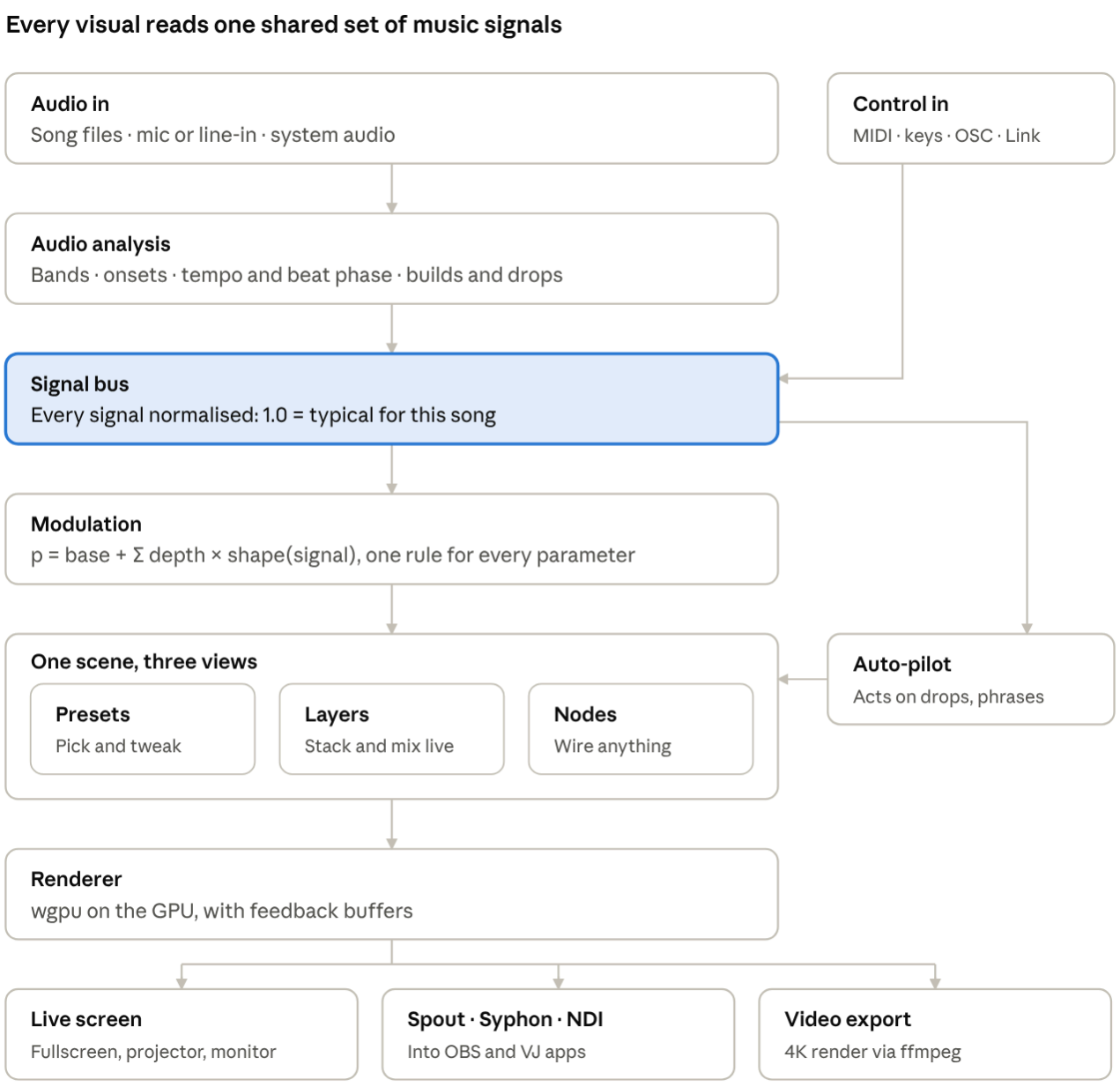
$$\theta(t) = \int_0^t v_0\left(1 + a\,\ell_{\text{slow}}(u)\right) du$$

Colour follows brightness of sound. The spectral centroid is the balance point of the spectrum. Mapping it on a log scale to hue makes bright, hat-heavy sections shift colour and bass-heavy sections settle into deep tones:

$$C = \frac{\sum_k f_k\,|X[k]|}{\sum_k |X[k]|}, \qquad \text{hue} = \frac{\log_2(C / C_{\min})}{\log_2(C_{\max} / C_{\min})}$$

One engine, three ways to control it

Presets, layers and nodes are three views of the same scene, not three separate systems. That's what keeps everything consistent: a preset is a saved scene, a layer is a node chain shown as a stack, and every parameter in all three uses the same modulation rule.



engine architecture · audio in to outputs

Audio becomes one shared set of signals; your controls and the auto-pilot feed in alongside them, and one renderer sends the result to screen, other apps or a video file.

- **Presets:** pick a look, turn a few macro knobs. Best for quick use and auto-pilot.
- **Layers:** stack scenes and effects with blend modes and opacity, Resolume-style, and mix them live.
- **Nodes:** open any layer to see and rewire the graph underneath, TouchDesigner-style. Changes show up straight away in the other two views.
- **Scene format:** each building block is a shader with a small header declaring its controls, like ISF. Shadertoy and ISF shaders can be imported, and anything you build is one shareable file.

Platform and tech stack

Decision: a native desktop app, Windows and Mac, written in Rust with wgpu. You were right to push back: for live performance the browser loses on almost every requirement that matters.

Requirement	Desktop app	Browser
Fullscreen on a second monitor or projector	Full control	Limited, one window per tab
Spout, Syphon and NDI output to OBS and VJ apps	Yes	No
Capture system audio (whatever's playing)	Yes	Very limited
MIDI controllers	Yes, everywhere	Not in Safari
Render a 4K video export faster or slower than real time	Yes, via ffmpeg	Hard and slow
Never throttled or paused in the background	Yes	Tabs get throttled
Share with one link	No	Yes

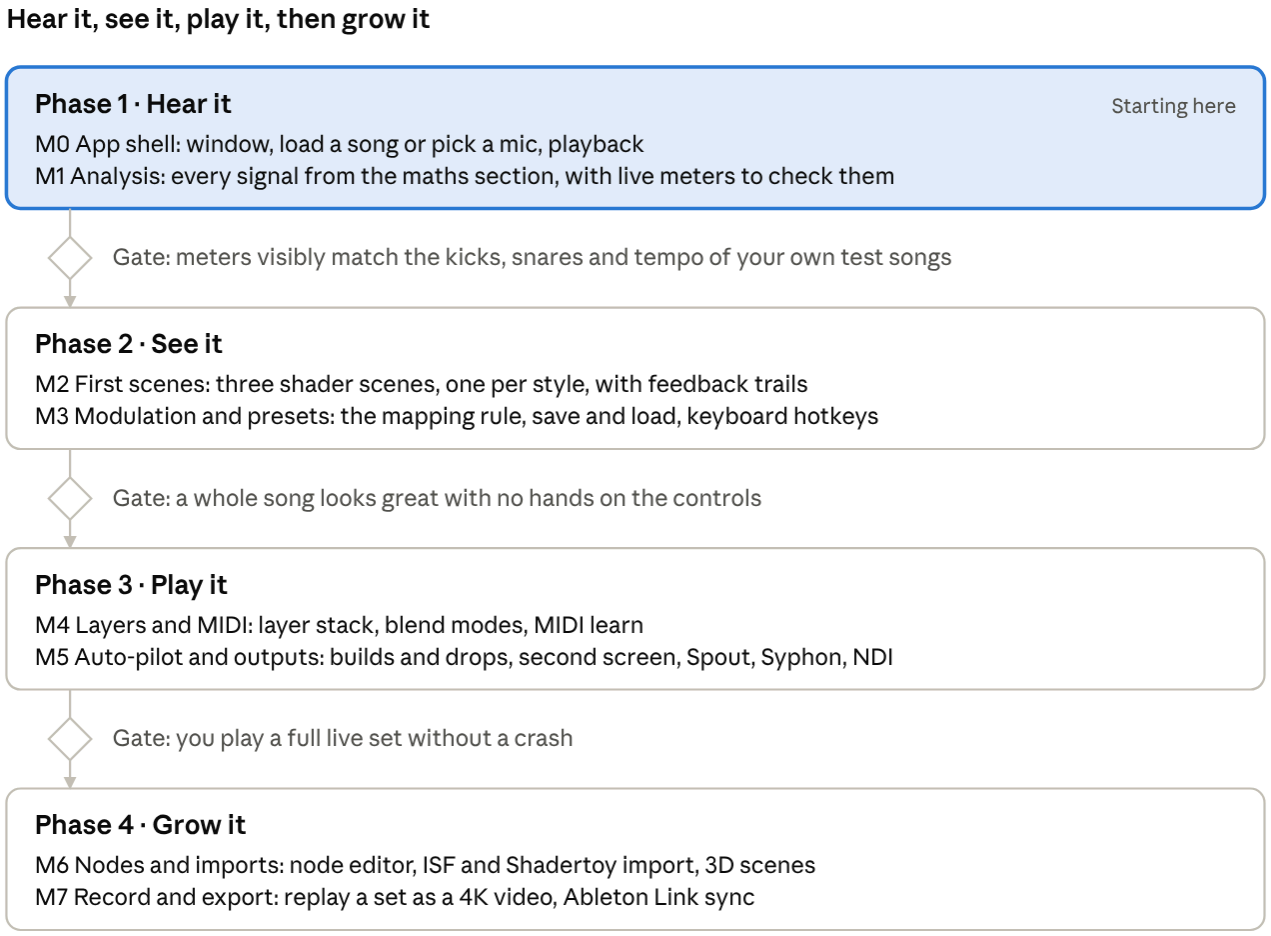
Why Rust and wgpu:

- wgpu runs natively on Vulkan, Metal, DirectX 12 and OpenGL ES, and can also compile to WebGPU in a browser. One engine could later power a lightweight web player for sharing, without building it twice.
- wgpu includes naga, which translates shaders between GLSL, WGSL and others. That makes importing Shadertoy and ISF shaders realistic.
- Rust's compiler rejects whole classes of crashes before the app ever runs. That matters when Claude writes the code and you test it: fewer mystery crashes mid-set.
- Mature libraries exist for everything else: cpal (audio in and out), realfft (FFT), symphonia (MP3/WAV/FLAC decoding), midir (MIDI), rosc (OSC), egui (the interface) and ffmpeg (video export). There's also a Rust Ableton Link implementation, though it's early-stage.

Alternatives considered: Godot and Unity are game engines and add weight we don't need. C++ with openFrameworks is proven but more crash-prone to debug. TouchDesigner is excellent but you'd be building inside someone else's product.

Roadmap

The analysis comes first, because every visual depends on it; if the signals are wrong, nothing will look right. Each gate is a test you run before moving on.



roadmap · 4 phases, 8 milestones, 3 gates

There are no dates yet. Every milestone ends with an app you can run. Milestone 0 is the first job for Claude Code once the open questions below are answered.

Open questions

- [ ] Is your main computer Windows or Mac, and what graphics card does it have? This sets the first build target.
- [ ] Do you own a MIDI controller? If so, which one, so it can be the first one mapped.
- [ ] Open source or private? Open source makes Ableton Link free to use and invites shader contributors.
- [ ] What should it be called?
- [ ] Where is the GitHub repository, and are you OK installing Rust (one installer)?

Sources

- Resolume: Parameter animation and audio analysis
- Resolume Avenue & Arena
- MilkDrop preset authoring guide
- MilkDrop on Wikipedia
- MilkDrop preset example, Fire and Brimstone
- Synesthesia scenes (Victor Shepardson)
- Using ISF
- ISF in VDMX
- Hydra on GitHub
- Audio-reactive visuals in TouchDesigner
- Ableton Link
- Ableton Link source and licence
- ableton-link-rs
- NDI and Syphon/Spout explained
- wgpu and wgpu overview
- OBTAIN: Real-time beat tracking (spectral flux)
- @audio/beat: onset, tempo and beat tracking reference
- SuperFlux onset detection (Böck and Widmer, DAFx-13)